



Share a seat. Split the fare.

Never overbook.

A transaction-safe, explainable ride-pooling engine for Dhaka's three-seat auto-rickshaws ("Teslas"). This document covers the problem, the design decisions and the evidence behind them.

► Live app: teslapool.vercel.app

github.com/Emon3469/TeslaPool

0

capacity violations,
enforced by the database
itself, not just the code

326

automated tests: 240
backend · 54 frontend unit
· 32 browser end-to-end

₳0.01

fare precision: integer
poysha, verifiable by hand

100%

of match, reject and fare
decisions explained in plain
language

CONTENTS

01	Executive summary	08	Concurrency: the last-seat problem
02	The core problem: root causes, stakeholders, pain points	09	Ride lifecycle and data integrity
03	Existing solutions and gap analysis	10	Architecture and system design
04	The core differentiator and key innovations	11	Machine learning, used only where it belongs
05	The matching algorithm	12	Security, quality and verification
06	The fare engine and how we evaluate it	13	Scaling to 1M passengers
07	Jashim's decision: the driver problem	14	Trade-offs, limitations and roadmap

Executive summary

In Dhaka, millions of short trips run on three-seat auto-rickshaws. Riders going the same way pay for a whole vehicle or bargain on the street, and drivers leave seats empty. Pooling is the obvious fix. It fails in practice for three reasons: **nobody trusts the price**, **nobody understands the match**, and **the software can overbook the vehicle**.

TeslaPool in one sentence: a ride-pooling engine that explains why two passengers were matched, shows each passenger exactly how their fare was calculated, and guarantees that a vehicle's seats can never be exceeded, even when two people tap "join" at the same millisecond.

The story the whole system is built around

8:41 AM, Banani Road 11. **Jashim** waits beside **Bullet**, his three-seat auto-rickshaw. **Nusrat** books a ride to Mohakhali. Two minutes later **Rafiq** books a ride to Gulshan 1, which is almost the same direction. **Arif** books Mohakhali too. Then **Shirin** tries for a seat that no longer exists.

What TeslaPool decides, in milliseconds

- Nusrat and Rafiq can share Bullet via **Banani** → **Gulshan 1** → **Mohakhali**. Dropping Rafiq first adds only 1.1 km to Nusrat's trip.
- The reverse order is **rejected**: it would add 2.9 km to Rafiq's 2.5 km trip.
- Each rider pays a **different, individual** fare based on how much of *their own* trip was shared.
- Shirin is refused with a reason: *"Only 0 seats left, 1 requested."*

What makes it trustworthy

- **Explainable**: every decision stores a headline and one plain-language reason per rule.
- **Race-safe at two layers**: row locks in the application *and* a capacity trigger in PostgreSQL.
- **Exact money**: integer poyssha, itemised, frozen per ride, checkable with a calculator.
- **Auditable**: append-only ride history that the database refuses to rewrite.

What was built

Surface	Capabilities
Passenger app	Sign-up with email verification, ride requests with a live fare preview and urgency choice, explainable pool options, one-tap join or auto-match, live ride page with timeline, cancellation, history, TeslaPay wallet
Driver app	Go online/offline, live pool with stop order and per-rider fares, ranked and explained waiting requests, accept, arrive → start → complete, vehicle management, trip history
Ops console	Live pools with seat occupancy, ride oversight and cancellation, integrity KPIs recounted from raw rows
Intelligence	Prediction lab (ETA + fare with guardrail band), engine insights, live system health (API, database, ML)
Public site	Landing page, fare calculator, how it works, safety, live impact dashboard, service areas map, newsroom, legal

Platform

REST API with a generated OpenAPI contract, 8 migrations with hand-written constraints and triggers, Docker images, Render + Vercel deployment, ML sidecar

The core problem we solve

Dhaka is one of the most congested cities in the world. Short trips of 2–6 km are the city's everyday unit of movement, and the auto-rickshaw is its workhorse. Yet a three-seat vehicle routinely carries one passenger, while two more people going the same way stand at the same corner.

Root causes

Pooling fails because of five root causes. Each one maps to an engineering decision later in this document.

#	Root cause	Why it blocks pooling	Where we address it
1	Price opacity	Street fares are bargained; app fares are one opaque number. A rider who is told "sharing is cheaper" can't check by how much, or why a co-rider pays less.	\$6 Fare engine
2	Match opacity	Riders don't know why they were paired with a stranger, or why a pool refused them. Unexplained refusals feel arbitrary; unexplained detours feel like being cheated.	\$5 Matching
3	Capacity is a hard physical limit, software treats it as a counter	Two riders racing for the last seat can both be "confirmed" by naive code. That leaves a double-booked seat, an argument on the street, and lost trust.	\$8 Concurrency
4	Routes overlap but rarely match	Real riders share a direction, not an exact origin and destination. Matching only identical routes finds almost nobody; matching everything creates huge detours.	\$5 Matching
5	Drivers decide under uncertainty	A driver with a partly full vehicle doesn't know which waiting rider fits the route, what they will pay, or whether a new rider hurts current passengers.	\$7 Jashim's decision

Stakeholders

Passengers (Nusrat, Rafiq, Arif, Shirin)

Want a lower fare, a predictable arrival and a fair deal. **Need:** proof the shared price is honest and a clear answer when they can't join.

Drivers (Jashim + Bullet)

Want more earnings per km and fewer empty seats, without angry passengers. **Need:** a ranked, explained queue and a vehicle that can't be overbooked.

Operators (TeslaPool Ops)

Want pooling that is measurably safe and profitable. **Need:** live integrity KPIs, audit history and the power to intervene.

The city

Fewer vehicle-km for the same trips means less congestion and emissions.

Need: higher occupancy per vehicle.

Evaluators and regulators

Want to verify that prices are fair and nobody is discriminated against. **Need:** fares computable by hand and decisions reconstructable after the fact.

Engineering team

Wants to change rules and prices without breaking guarantees. **Need:** a pure domain core, configuration-driven rules and an exhaustive test suite.

Current pain points

Who	Pain point today	TeslaPool's answer
Passenger	"Why am I paying this?" The discount is a mystery and can't be verified.	Itemised fare: base + distance + time sums exactly to the total; the discount is derived from <i>your</i> shared distance
Passenger	"Why was I paired with them?" / "Why can't I join?"	Every rule reported with a ✓ or X and a sentence, plus the rejected route alternatives
Passenger	Double bookings, and co-riders seeing personal data	Two-layer capacity guarantee; co-riders see only first names and zones
Driver	Empty seats; no way to know which waiting rider fits the route	Compatible requests ranked: feasible first, then route score, then longest wait
Driver	Disputes about the fare at drop-off	Fare frozen at pickup; cash or wallet settled in the same transaction as completion
Operator	"Impact" claims with no evidence	<code>/stats/impact</code> : every KPI recomputed live from raw database rows
Everyone	Retries, double taps and flaky networks create duplicates	Idempotency keys, plus unique indexes: one active ride per passenger, one active membership per ride

Existing solutions and gap analysis

Ride pooling isn't new. What's missing is pooling that riders can **verify**, drivers can **act on**, and operators can **prove** is safe. The table compares solution categories rather than individual companies, because the gaps are structural.

Category	Examples	What works	Gap
Solo ride-hailing	Uber, Pathao, Obhai	On-demand booking, upfront price, tracking	One rider per vehicle. Empty seats are waste, and the price is a single opaque number.
Negotiated-fare apps	inDrive-style bidding	Rider has price agency; drivers choose jobs	Price depends on bargaining power, not distance. No pooling, and no guarantee two riders are priced consistently.
Global pooling products	UberPool, later UberX Share	Proven demand for shared rides at a discount	Detours and discounts are a black box. Riders can't see why a match happened or how their share was computed.
Informal street sharing	Shared CNGs / legunas on fixed lines	Very cheap, culturally familiar	Fixed routes only. Fares are set by custom, with no booking, safety record or capacity control. The driver waits until the vehicle fills.
Academic dial-a-ride solvers	VRP / DARP optimisers	Near-optimal routing at fleet scale	Optimise a global objective without per-rider explanations. They assume a consistent state and don't address concurrent booking races.

How TeslaPool fills each gap

Gap	How we fill it	Evidence
Opaque shared price	Deterministic, itemised formula; discount from the rider's <i>own</i> shared fraction; integer money; breakdown frozen per ride	By-hand test reproduces Nusrat's and Rafiq's fares to the poysha
Black-box matching	Six documented hard rules, each reported with a sentence; the rejected alternative stop orders are shown too	Unit tests per rule; <code>GET /rides/:id/explanation</code>
Overbooking under concurrency	Pessimistic row lock with re-validation <i>and</i> a database trigger that recounts the real seat sum	10 concurrent racers → exactly 1 winner; mutation test (§8)
Fixed routes only	Every pickup-before-drop-off stop order is enumerated; detour is bounded <i>for every passenger</i> , not on average	Route-planner tests; the Banani → Gulshan → Mohakhali example
Driver guesswork	Ranked, explained candidate queue; accept runs the same transactional join; unsafe actions are impossible	Driver e2e tests; forced accept returns <code>409 CAPACITY_EXCEEDED</code>

Unprovable impact

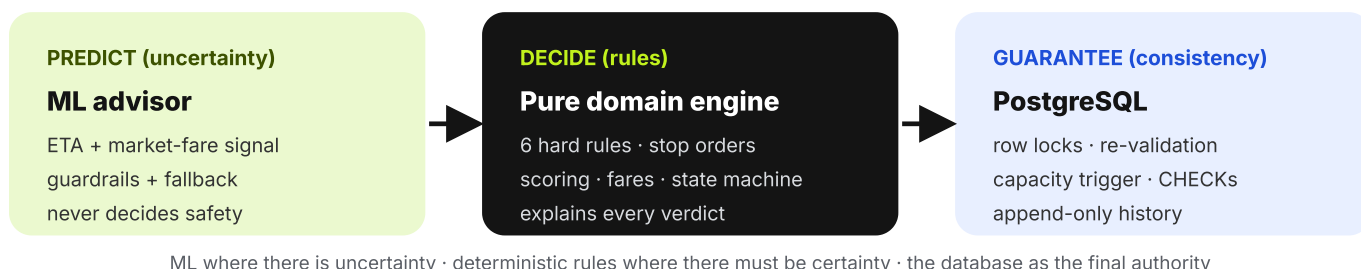
KPIs recounted live from raw rows,
including an integrity query for capacity
violations

GET /stats/impact →
integrity.capacityViolations
= 0

The core differentiator

Every promise TeslaPool makes to a rider is either explained in plain language or enforced by the database, and usually both.

Other pooling products ask riders to trust the algorithm. TeslaPool shows its work. “You pay ₹108.93 because 55.6% of your 4.5 km trip was shared, earning a 13.89% discount on a ₹126.50 solo fare” is a sentence a rider can check. “Bullet cannot carry a fourth passenger” is a sentence PostgreSQL enforces, even if the application code has a bug.



ML where there is uncertainty · deterministic rules where there must be certainty · the database as the final authority

Key innovations / USPs

USP 1 Explainable pool engine

Each match decision carries a headline and a ✓/✗ sentence for every rule, plus the rejected stop orders and why they failed. Rejections are stored as `MATCH_REJECTED` events. That builds the labelled dataset a future ranking model needs, which no public dataset has.

USP 2 Two-layer capacity guarantee

Layer 1: `SELECT ... FOR UPDATE` on the pool, then the ride (fixed lock order, no deadlocks), and the full engine re-run on fresh state. Layer 2: a trigger that locks the pool and refuses any membership whose *real* seat sum would exceed capacity. Proven by mutation testing.

USP 3 Individual, hand-checkable fares

Each passenger's discount comes from the fraction of *their own* distance ridden with someone. Integer poysha, integer basis points, explicit half-up rounding. Components always sum exactly to the total, and the breakdown is frozen on the ride.

USP 4 Driver decision support

Jashim sees waiting riders ranked by feasibility, route score and wait time, with the *same verdict* the passenger sees. He can accept; he can't overbook, go offline with passengers aboard, or skip lifecycle steps.

USP 5 Immutable, narrated history

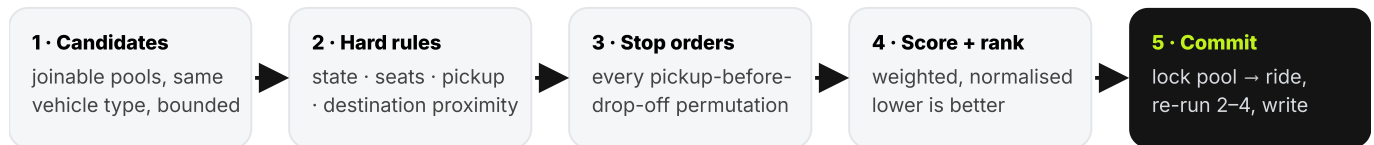
Ride and prediction events are append-only; a trigger rejects UPDATE and DELETE. The ride page narrates the timeline: “*Rafiq joined; your fare ₹126.50 → ₹108.93.*” Every prediction stores its model version and exact feature snapshot.

USP 6 ML with guardrails, never in charge

ML estimates ETA and a market-fare signal inside plausibility bands (ETA within $0.5\times-2\times$, fare within $\pm 20\%$). A 1.5 s timeout and a circuit breaker back them up. If the sidecar dies, rides are still created, matched and priced; readiness reports `ready_degraded`.

The matching algorithm

Matching is a pipeline: **generate candidates** → **check hard constraints** → **enumerate stop orders** → **score** → **commit under a lock**. The first four steps are pure functions: no I/O, no clock, no randomness, and rules passed in from configuration. That makes them deterministic, unit-testable and safe to re-run inside a transaction.



Steps 1–4 are advisory: no locks held, fast to compute. Step 5 is authoritative: the transaction re-evaluates on fresh state and may still say no.

The six hard rules (all must hold; all are reported)

#	Rule	Default (configurable)	Reason message when it fails
1	Pool is joinable (not started)	OPEN or MATCHED	"Trip has already started; no new passengers can join"
2	Free seats ≥ requested seats	capacity 3	"Only 0 seats left, 1 requested"
3	New pickup within 1 zone hop of <i>every</i> current pickup	PICKUP_MAX_HOPS=1	"Pickup is 2 zones from current passengers (limit 1)"
4	New destination within 2 hops of <i>every</i> current destination	DESTINATION_MAX_HOPS=2	"Destination is 4 zones from current passengers (limit 2)"
5	Some stop order has ≤ 4 distinct stops	MAX_STOPS=4	"Every route would need more than 4 stops"
6	In that order, every passenger's detour ≤ their own limit (standard max(1.5 km, 30%), urgent max(0.5 km, 10%), flexible max(3 km, 60%)). Detour = waiting while the car serves others before pickup + extra km in the car	MAX_DETOUR_*, URGENT_*, FLEXIBLE_*	"Every possible route adds too much detour for someone", with each rejected stop order listed

Waiting counts. Detour includes the distance the car drives before picking you up while it serves others. Otherwise the engine could "pool" by dropping one rider and driving back for the next: zero km in the car, a 5 km wait.

Rule 6 is checked for **every** rider, existing and new, not on average. Averages hide victims: a plan where one rider absorbs a 3 km detour for everyone else's benefit is rejected even if the mean detour looks fine.

Scoring: choosing among feasible plans

Only plans that pass every hard rule are scored. Each term is normalised to [0, 1], and the weights come from configuration, not from code:

$S = 0.35 \cdot \text{distance} + 0.25 \cdot \text{detour} + 0.15 \cdot \text{stops} + 0.10 \cdot \text{timeVariance} - 0.15 \cdot \text{sharing}$ (lower is better)

distance incremental vehicle-km this rider adds ÷ their solo km (1 = like a dedicated vehicle)
detour worst passenger detour ÷ that passenger's allowed detour
stops stop count ÷ MAX_STOPS
timeVariance std-dev of per-passenger detour ratios (fairness: nobody absorbs all the detour)
sharing mean fraction of in-vehicle distance ridden together (the pooling benefit)

Why a weighted score and not an optimiser? With at most 3 seats and 4 stops, a full enumeration is tiny (a handful of permutations), so we get the exact best plan without heuristics. Each term also means something a person can read. The `timeVariance` term is the one most solutions leave out: it penalises plans that are efficient on average but unfair to one rider.

Worked example: Nusrat and Rafiq

ACCEPTED Banani → Gulshan 1 → Mohakhali

Rafiq rides 2.5 km, his solo distance, so his detour is 0.
Nusrat rides $2.5 + 2.0 = 4.5$ km against a 3.4 km solo trip, a **1.1 km** detour. Her limit is $\max(1.5, 30\% \times 3.4 = 1.02)$, which is 1.5 km. ✓ **within limit**

REJECTED Banani → Mohakhali → Gulshan 1

Nusrat rides 3.4 km, her solo distance.
Rafiq rides $3.4 + 2.0 = 5.4$ km against a 2.5 km solo trip, a **2.9 km** detour. His limit is $\max(1.5, 30\% \times 2.5 = 0.75)$, which is 1.5 km. ✗ **DETOUR_TOO_HIGH**. The UI shows this order as the rejected alternative.

Urgency: when Nusrat is late

Each rider picks **Urgent**, **Standard** or **Flexible**. The choice changes only *their own* detour limit, and every order must still satisfy every rider.

Nusrat (urgent) with...	Result
Rafiq flexible (limit 3 km)	Banani → Mohakhali → Gulshan 1. Nusrat: 0 km extra. Rafiq: 2.9 km extra, within his limit.
Rafiq standard (limit 1.5 km)	Not matched: no order keeps Nusrat direct without detouring Rafiq 2.9 km. He waits for another pool, with the reason shown.

Why this is fair: urgency never borrows from anyone else's limit, and it has a price, because fewer pools fit an urgent rider. So it can't be used to jump the queue. Among riders who fit, the driver sees urgent ones first.

Traffic: the model uses one city-wide traffic level, which slows every stop order equally, so it can't change which order is best. Per-road traffic would come from a real routing provider behind `DistanceProvider`; the rules wouldn't change.

Auto-match: best plan first, with a safe fallback

`POST /rides/:id/match` ranks candidate pools (feasible first, then lowest score) and tries up to three of them **transactionally**. If another rider takes the last seat between ranking and commit, the transaction's fresh evaluation rejects the join, the attempt is counted as a *lost race*, and the next pool is tried. The rider never sees a stale "you're matched" that turns out to be false.

Geography is a documented demo abstraction: 10 Dhaka zones (Banani, Gulshan, Mohakhali, Uttara, Mirpur, Dhanmondi, Farmgate, Azimpur, Bashundhara R/A, Motijheel), an adjacency graph for hops, and approximate road distances. The engine depends only on a `DistanceProvider` interface, so a real routing service can replace the graph without changing a rule.

The fare engine and how we evaluate it

The brief required that an evaluator can check each passenger's fare **by hand**. That one requirement shaped the whole pricing design: the price is a transparent formula, money is never a floating-point number, and ML may *advise* but not set the price by default.

```

soloFare = base + distanceCharge + timeCharge
          base ₳50 · distance ₳20/km · time ₳0.50/min
          minutes = km × min-per-km for the traffic level
          (LOW 3.3 · MEDIUM 5.0 · HIGH 7.5 · GRIDLOCK 12.0, medians from the Dhaka
dataset)

sharedFraction = share of YOUR in-vehicle distance ridden with at least one other passenger
poolDiscount   = min(25%, 25% × sharedFraction)
passengerFare  = soloFare × (1 - poolDiscount)          rounded half-up to a whole poysha
  
```

Worked example (medium traffic), verified to the poysha by an automated test

	Nusrat · Banani → Mohakhali	Rafiq · Banani → Gulshan 1
Solo distance and time	3.4 km · 17.0 min	2.5 km · 12.5 min
Solo fare (poysha)	5000 + 6800 + 850 = 12650 (₳126.50)	5000 + 5000 + 625 = 10625 (₳106.25)
Shared on Banani → Gulshan 1 → Mohakhali	2.5 of 4.5 km = 55.56%	2.5 of 2.5 km = 100%
Pool discount	25% × 0.5556 = 13.89%	min(25%, 25%) = 25%
Passenger fare	12650 × 0.8611 = 10892.9 → 10893 (₳108.93)	10625 × 0.75 = 7968.75 → 7969 (₳79.69)
When Arif joins (Banani → Mohakhali)	100% shared → 25% → 9488 (₳94.88)	unchanged → 7969

Why Nusrat and Rafiq pay different discounts, and why that is fair: Rafiq shares his entire trip. Nusrat rides the last 2 km alone after he gets off, so she gets the full benefit of sharing on only 55.6% of her trip. A flat “pool discount” would overcharge Rafiq relative to Nusrat. Tying the discount to each rider's *own* shared fraction is proportional and explainable.

How we evaluate the fare engine

Evaluation	What it proves	How
By-hand oracle	The deployed API returns exactly the fares a human computes with the formula above (Nusrat 12650 → 10893, Rafiq 10625 → 7969)	tests/e2e/prd-fare-by-hand.test.ts over HTTP
Itemisation invariant	base + distance + time always equals the total, to the poysha, for every quote	Unit tests; the browser e2e suite also checks the UI's breakdown against the API

Exact arithmetic	No floating-point money anywhere: km → integer metres and min → integer seconds, multiplied by integer rates, divided with explicit half-up rounding	<code>divRoundHalfUp</code> rejects non-integers; <code>fare-engine.test.ts</code>
Discount bounds	No sharing gives no discount; a fully shared ride gets exactly 25%; the discount is capped even if α is misconfigured, so a fare never drops below 75% of solo	<code>fare-engine.test.ts</code> (cap, scaling, integer application)
Stability over time	A ride's explanation never changes when prices are reconfigured later: the breakdown is frozen on the ride	Stored components on <code>ride_requests</code>
Lifecycle correctness	Fares re-price on every join or leave <i>until pickup</i> , lock at <code>STARTED</code> , and the settled payment equals the fare shown	Driver e2e: payment equals displayed fare
ML guardrail boundaries	Exactly +20% is inside the band and +20.01% is outside; a nonsensical or unavailable ML value falls back to the formula	<code>fare-engine.test.ts</code>
Market sanity (ML)	How far our policy price is from observed Dhaka market fares, and how often an ML estimate lands within $\pm 20\%$	Out-of-fold simulation: ML accepted for 51% of trips

Pricing mode is an explicit business switch. `FARE_PRICING_MODE=deterministic` (the default) charges exactly the formula, and the ML fare is logged and shown as an advisory signal. `ml_guarded` lets the model set the price, but only inside $\pm 20\%$ of the formula. The model never makes that choice on its own.

Why this is good for Jashim too

On the three-rider Bullet trip, the riders pay $\text{₹}94.88 + \text{₹}94.88 + \text{₹}79.69 = \text{₹}269.45$ for 4.5 vehicle-km, about **₹59.9/km**. A single Banani → Mohakhali solo trip earns $\text{₹}126.50$ for 3.4 km, about **₹37.2/km**. That is $\approx 61\%$ **more revenue per km** for the driver, while the riders together save **₹89.80**. Pooling done right is positive-sum.

Jashim's decision: the driver problem

Jashim has three seats, a route that changes with every rider who joins, and a queue of strangers asking to get in. He has to decide **whom to accept, when to stop accepting and when to go**. Every one of those choices can make a passenger's fare or trip time worse, or break the vehicle's capacity.

The decision, broken down

Jashim's question	What TeslaPool shows or does
"Who is waiting that fits my route?"	His dashboard lists compatible waiting requests, evaluated by the same engine passengers see, against his pool's current riders and planned route.
"Who should I take first?"	Ranked: (1) feasible first → (2) best route score → (3) longest wait . Each card shows the rider's first name, zones, seats, wait time, verdict and fare.
"Will this rider hurt my current passengers?"	A request is feasible only if <i>every</i> current passenger's detour stays within their limit and stops stay ≤ 4 . Co-riders' fares are re-priced automatically as the shared distance changes.
"Can I squeeze in one more?"	No. When Bullet is 3/3, Shirin is shown with the verdict <i>"Not matched: Only 0 seats left, 1 requested"</i> and no Accept button . A forced accept via the API returns <code>409 CAPACITY_EXCEEDED</code> .
"Can I log off now?"	Not while passengers are assigned. Going offline is refused, so riders are never stranded.
"When do I go?"	Arrived at pickup → Start trip → Complete trip. Start locks fares and stops new joins; Complete settles cash or wallet payment in the same transaction.

Why this is the correct design

1 • One source of truth

The driver's view and the passenger's view are produced by the **same pure function**, `evaluateCandidate()`. Jashim can never see "fits" while Shirin sees "full". Disagreement between sides is structurally impossible, not just unlikely.

2 • Human agency, machine safety

Jashim keeps the decision (accept or not), because he knows the street better than any model. The system removes only *unsafe* options: overbooking, excessive detours, stranding riders, skipping steps. The driver gets a judgment call, not a trap.

3 • Fair to riders who wait

Among equally good matches, the longest-waiting rider comes first. That prevents starvation, where a rider on an awkward route is skipped forever. The ranking is transparent, so it can be audited for bias.

4 • Accept equals join, under the same lock

"Accept" doesn't bypass anything: it runs the identical transactional join (lock pool → ride, re-evaluate on fresh state). If a passenger self-joins the last seat at the same instant Jashim taps Accept, exactly one succeeds and the other gets a precise reason.

5 • Economically aligned

Pooling raises Jashim's revenue per km by about 61% on the demo trip (\$6). The detour limits and the fairness term protect riders, so the driver's incentive to fill seats never works against the passengers already aboard.

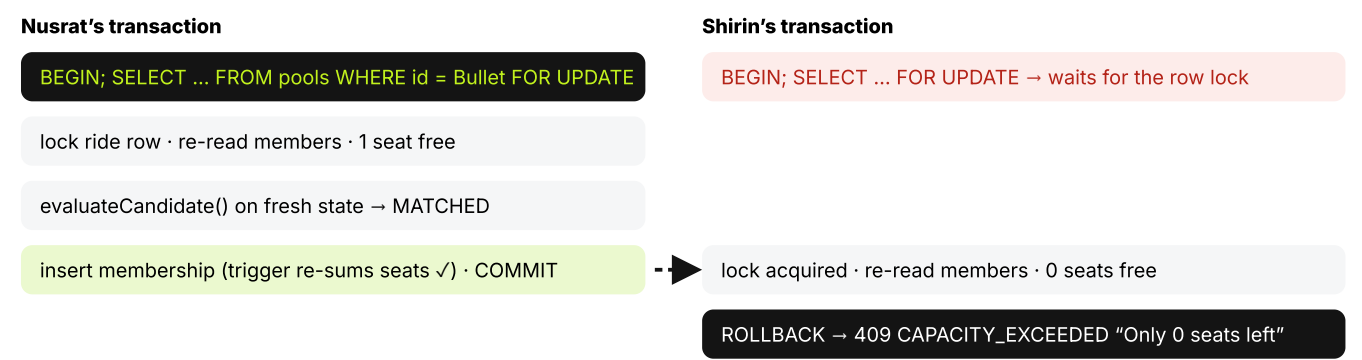
6 • Verified end to end

Browser e2e tests log in as Jashim and accept a rider (the API confirms `MATCHED`). They run a full trip where the settled payment equals the fare shown, then confirm the 3/3 scenario: Shirin is refused, her card has no Accept button, and ops sees 0 capacity violations.

Jashim's online status isn't a separate flag that could drift out of sync: **online** $\Leftrightarrow$ **he has a live pool**. That removes a whole class of "driver shows online but can't receive riders" bugs.

Concurrency: the last-seat problem

Bullet has one seat left. Nusrat and Shirin both see it and tap “join” at the same instant. A naive *read seats → check → write* lets both of them read “1 free” and both write. The result: four people for three seats.



The second transaction blocks on the pool row until the first commits, then re-evaluates on fresh state. It gets a precise, explainable refusal, not a silent overbooking.

Two independent layers

Layer 1: application (pessimistic locking)

- SELECT ... FOR UPDATE on the pool, then the ride. A global lock order prevents deadlocks; a rare deadlock is retried.
- The **full matching engine re-runs** inside the transaction. The pre-lock ranking is only advice.
- No network call is ever made while a lock is held. ML predictions happen before the transaction.

Layer 2: database (enforced by PostgreSQL)

- A trigger on pool_memberships locks the pool and refuses any insert whose **real** seat sum would exceed capacity.
- occupied_seats is maintained by the database, not written by the app.
- A partial unique index allows only one *active* membership per ride.

Proof by mutation testing

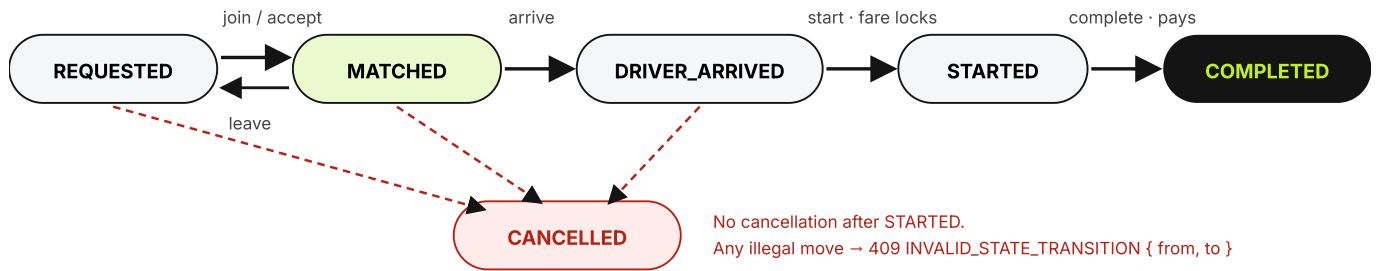
Experiment	Result
10 concurrent requests for 1 last seat, both layers on (regular suite)	exactly 1 winner, 9 precise refusals
Application lock removed, only a CHECK (occupied_seats ≤ capacity)	7 of 10 “won” the same seat: 8 riders in 6 seats. The CHECK passed, because every racer wrote the same stale counter.
Application lock removed, capacity trigger + DB-owned counter on	correct: the database alone refuses the overbooking

This experiment is why Layer 2 exists. It turns “the code is careful” into “the database refuses”. The lesson generalises: a **CHECK constraint on a counter is not a concurrency control**, because it validates the value written, not the truth.

Why pessimistic, not optimistic, locking?

The last seat is a *hot row*: contention on it is expected, not rare. Optimistic retries would make every racer do the work and then most of them retry, which is wasteful and non-deterministic. `FOR UPDATE` serialises contenders cheaply, and contention is **per vehicle** (≤ 3 seats), never global. So the design scales with the number of vehicles, not the number of riders.

Ride lifecycle and data integrity

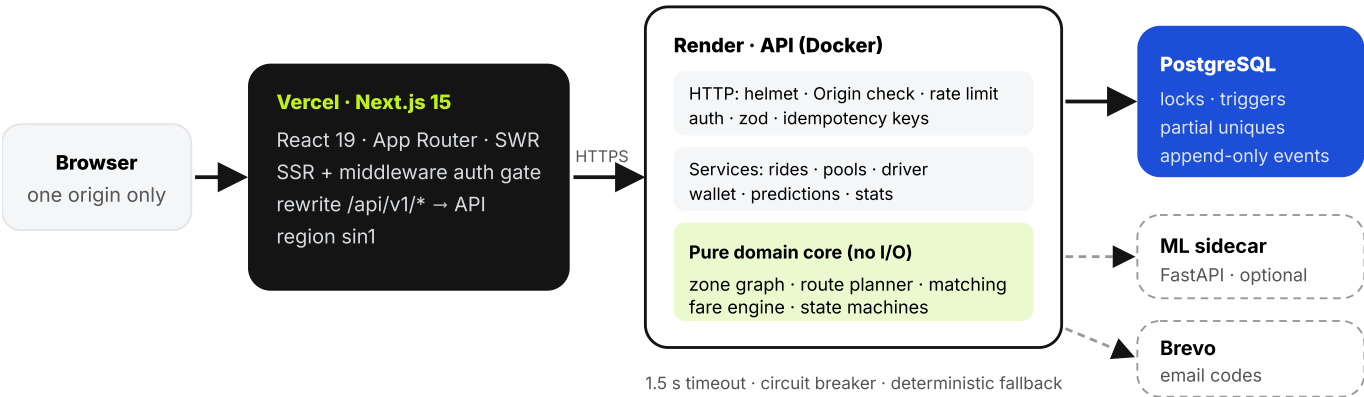


- **Explicit state machine**, exhaustively tested over all 36 state pairs. No generic `PATCH status` endpoint exists, so there is no way around it.
- **Derived pool status**: a pool's state is computed from its riders' states inside the same locked transaction, so a pool can never say `MATCHED` while its rider is `STARTED`.
- **One deliberate addition** to the suggested lifecycle: `MATCHED → REQUESTED`, which lets a rider leave a pool before pickup and re-match without cancelling.

Integrity guarantees enforced by the database

Guarantee	Mechanism
Seats never exceed capacity	Capacity trigger (real seat sum) + DB-owned counter + CHECK
One active ride per passenger	Partial unique index on non-terminal rides
One active membership per ride	Partial unique index
History cannot be rewritten	Triggers reject UPDATE/DELETE on <code>ride_events</code> , <code>prediction_events</code> , <code>payments</code> , wallet ledger
Wallet can't overdraw; balance can't drift	Balance maintained by a trigger from a sign-checked, append-only ledger; one payment per ride (unique)
Retries don't duplicate effects	<code>Idempotency-Key</code> per user, payload-hashed (reuse with a different body → 422, in-flight → 409, 24 h TTL)
Money is exact	Integer poysha columns, integer basis-point discounts

Architecture and system design



Production topology. The browser only talks to the Vercel origin, so the session cookie is first-party (HttpOnly, Secure, SameSite=Lax) and the API URL never appears in client code. Both hosts run in Singapore, the closest region to Dhaka.

Key design decisions

Decision	Why	Trade-off accepted
Modular monolith	The hardest problem (the last-seat race) is solved by a single ACID transaction. Splitting pools and rides into services would turn it into a distributed-transaction problem for no benefit at this scale.	One deployable; module boundaries kept strict so a later split is possible
Pure domain core	No framework, database, clock or randomness in the engine. Deterministic, unit-testable in milliseconds, and reusable in future matching workers.	Services orchestrate I/O explicitly
PostgreSQL	Row locks, CHECKs, partial unique indexes , triggers and JSONB let the database <i>enforce</i> invariants. SQLite has no row-level locking; MySQL has no partial indexes.	Hand-written SQL in migrations
Same-origin proxy (BFF)	Next.js rewrites <code>/api/v1/*</code> to the API, so there's no browser CORS, the cookie is first-party, and the backend URL stays server-side.	<code>BACKEND_URL</code> is baked in at build time; the build fails loudly if it's missing
zod as the single schema	One definition gives runtime validation (<code>.strict()</code> blocks mass assignment), TypeScript types <i>and</i> the OpenAPI contract, so docs can't drift from behaviour. The frontend uses types generated from that contract.	None found
REST with action endpoints	The domain is a lifecycle: <code>POST /rides/:id/start</code> maps 1:1 to transitions, with per-request idempotency and standard status codes.	Real-time is polling for now; SSE planned
Explanations from stored facts	Every decision and fare component is persisted, so history stays true after reconfiguration.	More storage per ride

Request flow: a passenger joins a pool

```

POST /api/v1/pools/:id/join      (browser → Vercel rewrite → Render)
├ request-id → helmet → rate limit → authenticate (JWT → reload user from DB) → Origin check
├ zod.strict() validation → idempotency key (per user, payload-hashed)
└ BEGIN
  SELECT pool FOR UPDATE → SELECT ride FOR UPDATE      -- fixed lock order
  re-read members · evaluateCandidate() on fresh state -- the transaction is the authority
  insert membership (capacity trigger re-sums seats) · re-plan route · re-price co-riders
  derive pool status · append ride_events (STATUS_CHANGED, ROUTE_UPDATED)
  COMMIT → 200 { decision, reasons[], fare, route }    | 409 with the failing rule's message

```

Data model (core tables)

users → vehicles → pools ← pool_memberships → ride_requests → ride_events (append-only) ·
 prediction_events (append-only, model version + feature snapshot) · payments (one per ride) ·
 wallet_transactions (signed ledger; balance set by trigger) · idempotency_keys . Money columns are
 integer poysha; fare components are frozen per ride. There are 8 migrations; the constraints and triggers
 that Prisma can't express are written by hand.

Machine learning, used only where it belongs

The rule is simple: **ML for uncertainty, rules for safety**. ETA and market fare are genuinely uncertain, and data helps there. Capacity, detour, ownership and state transitions are rules. A rule must give the same answer every time, be explainable, and be re-checkable inside a transaction.

Why there is no ML matching model, on purpose

The available dataset has fares, not match outcomes. It has no acceptance, rejection reason, detour tolerance or satisfaction label, so there is **no legitimate label** for match quality. Training on it would be pretending. Instead, TeslaPool **collects those labels**: every `MATCH_REJECTED` event stores the full explained decision. The future pipeline is already designed: candidate generation → hard-constraint filtering → *ML ranking* → deterministic validation inside the transaction.

Method: small data, handled honestly

- 100 real Dhaka trips (CNG auto-rickshaw 40, rickshaw 33, bike 27) across 10 zones. We treat this as a **prototype signal**, not production accuracy.
- `RepeatedKFold(5 × 3)`, 15 folds, reporting mean ± std. Deterministic baselines are evaluated on the *same* folds.
- **Leakage control**: actual trip duration is known only after the trip, so it is never a request-time feature. The fare model is trained on **out-of-fold ETA predictions**, so its training inputs look like serving inputs.
- Feature engineering lives in one module imported by **both** training and serving, so there is no train/serve feature drift.

ETA (request-time features only)

Model	MAE (min)	R ²
Ridge ✓ eta-v1	4.38 ± 1.13	0.976
GradientBoosting	5.45 ± 2.77	0.952
ExtraTrees	6.31 ± 2.74	0.937
Heuristic (km × min/km)	5.06	0.926

Fare (deployable: with predicted ETA)

Model	MAE (BDT)	R ²
ExtraTrees ✓ fare-v1	36.31 ± 11.32	0.935
GradientBoosting	36.92 ± 12.30	0.918
Ridge	43.58 ± 11.98	0.917
Leaky (actual duration)	30.70 X	not deployable

An honest finding: without distance × traffic interaction features, *every* ML model lost to the one-line heuristic. With them, Ridge beats it by about 13%, a real but modest gain. We ship it because it wins in cross-validation, and the heuristic remains the fallback. Reporting where ML *doesn't* help is as important as reporting where it does.

Guardrails, fallback and traceability

Situation	Behaviour
-----------	-----------

ML ETA outside 0.5×–2× the heuristic	Heuristic ETA used; ETA_GUARDRAIL_TRIGGERED
ML fare outside ±20% of the formula	Formula fare used; FARE_GUARDRAIL_TRIGGERED
Sidecar missing, slow, down or malformed	1.5 s timeout, circuit breaker (30 s), fully deterministic; readiness reports ready_degraded
Model or library mismatch	The sidecar refuses to start if its scikit-learn version differs from the model card, so a model is never silently swapped
“Which model priced this ride?”	Every prediction row stores model name, version, exact feature snapshot, latency and whether it was used

Security, quality and verification

Security controls

Threat	Control
Password theft	Argon2id (OWASP parameters); constant-work login (dummy hash for unknown emails)
XSS token theft	JWT in an HttpOnly, Secure, SameSite cookie; Bearer for scripts
CSRF	Cookie-authenticated writes need an allow-listed Origin → else 403
JWT tampering	HS256 pinned, <code>iss / aud / exp</code> checked; the role is reloaded from the DB, not trusted from the token
IDOR	Object-level checks on every ride, pool and vehicle; co-riders see first name + zones only
Mass assignment	<code>zod.strict()</code> ; prices and surge always computed on the server
Brute force / abuse	Rate limits per IP and per user; email codes stored as HMACs, 5 atomic attempts, 10-minute expiry
Account enumeration	Forgot-password always returns 202
Leaky errors	Uniform error envelope; no stack traces or SQL in responses (tested)
Container	Multi-stage images, production dependencies only, non-root users

Test pyramid (all green)

Suite	Scope	Count
Backend unit	Pure domain: route planner, matching, fares, state machine (all 36 pairs), explanations, config	240
Backend integration	Real PostgreSQL: auth security, rides, pools, payments, driver, ML fallback, email, rate limits, frontend contract	
Backend e2e	The full demo story + the fare-by-hand oracle over HTTP	
Concurrency	Last-seat races, double-pool races, idempotent replays, DB-only guarantees	
Frontend unit	API client, fare, geo, ride, format, stats; UI components (Vitest)	54
Browser e2e	Playwright against the live API: auth, passenger, driver, ops, public pages. Each UI number is compared with the API's answer	32

The browser suite treats the API as the **oracle**. It doesn't just check that a price appears; it checks that the price shown equals the API's `amountPoysha`, formatted by an independent copy of the formatter.

Supply chain and release

- `npm audit` : 0 vulnerabilities (backend and frontend)
- Git flow: feature branches → `pre-release` → `release/*` → `master`, conventional commits, tagged releases
- Fresh-clone builds verified before release (backend Docker build steps, frontend production build)

Measured, not asserted: live KPIs

`GET /api/v1/stats/impact` computes every KPI from raw rows: match success rate, p50/p95 matching latency, average saving per pooled passenger, seat utilisation, **capacity violations recounted from raw memberships**, occupancy-counter drift, invalid transitions rejected, lost races and guard hits. The public *Impact* page and the ops console read the same endpoint.

Scaling to 1M passengers and 100k drivers

Size the problem first, then choose the boxes. The transactional write load is modest; the heavy traffic is reads and ephemeral location updates, which shouldn't touch the primary database at all.

Quantity	Assumption	Result
Rides per day	20% of 1M ride daily, 1.2 rides each	≈ 240k / day
Peak ride creation	15% of daily rides in the peak hour	≈ 10 req/s (spikes ×5 → 50/s)
Matching operations	~3 evaluations + 1 transactional join per request	40–200 ops/s
Status reads	30k active rides polling every 5 s	≈ 6k reads/s
Driver GPS (future)	30k online drivers, 1 update / 4 s	≈ 7.5k writes/s → Redis geo, not Postgres

Concern	Evolution	What stays the same
API	N stateless instances behind a load balancer; rate limits move to Redis	Stateless JWT design (already)
Matching	Queue-fed workers partitioned by city or zone, batch-evaluating candidates	The pure engine and the locked join, reused unchanged
Geo search	H3 cells or PostGIS; candidates = same + neighbouring cells	Zone graph → cell graph behind <code>DistanceProvider</code>
Database	Primary for writes, read replicas for history and dashboards, monthly partitions for event tables	Capacity is a per-row invariant; it scales with the data
Contention	Lock timeouts + jittered retries	Contention is per vehicle (≤ 3 seats), never global
Real-time	Transactional outbox → queue → SSE/WebSocket gateway; removes ~6k polling reads/s	Append-only <code>ride_events</code> is already the event log
Observability	OpenTelemetry traces, Prometheus metrics, SLO alerts per city	The KPIs and events already exist

Principle: degrade features, never correctness. Each scaling step is triggered by a measured bottleneck (lost-race rate, p95 time-to-match, lock-wait time), not by an architecture diagram.

Trade-offs, limitations and roadmap

Deliberate trade-offs

Choice	Benefit	Cost we accept
Deterministic matching, not ML	Explainable, re-checkable in the transaction, and consistent	Not globally optimal across the fleet
Pessimistic locks	Deterministic, cheap for hot rows	Transactions must stay short (no network inside)
Fares re-priced until pickup	Riders benefit instantly when someone joins	A co-rider leaving before pickup can raise the fare again
Zone graph, not maps	Deterministic and testable, with no external dependency	Distances are approximations (clearly labelled)
Deterministic price by default	Checkable by hand, as required	Less market-adaptive than an ML price

Known limitations (stated honestly)

- Zone-graph distances are approximations, not GPS routing, which causes train/serve skew for the ML models.
- The ML dataset has ~100 rows: a prototype signal, and advisory by default.
- Payments are simulated (TeslaPay wallet and cash); there are no cancellation fees or refunds yet.
- Rate limiting is in-memory per instance; there are no refresh tokens yet (1 h access tokens, user reloaded per request).
- On the free hosting plan, the API sleeps after 15 minutes idle, so the first request takes about a minute.

Roadmap

Next

- SSE ride updates fed by `ride_events`
- Real routing behind `DistanceProvider`
- bKash / Nagad settlement

Then

- Match-ranking model trained on collected `MATCH_REJECTED` labels
- Conformal prediction intervals for ETA
- Refresh-token rotation

At scale

- Zone-partitioned matching workers
- Outbox + realtime gateway
- Redis geo index for live drivers

Try it: teslapool.vercel.app. Use the one-click demo logins on the login page. Fill Bullet with Nusrat, Rafiq and Arif, watch Shirin get refused with a reason, then open Jashim's dashboard to see the same verdict from the driver's side.

Source: github.com/Emon3469/TeslaPool